

AAA	未组合的十进制加法调整指令 AAA(ASCII Adjust for Addition) 格式: AAA 功能: 对两个组合的十进制数相加运算(存在 AL 中)的结果进行调整, 产生一个未组合的十进制数放在 AX 中.	说明: 1. 组合的十进制数和未组合的十进制数: 在计算中, 十进制数可用四位二进制数编码, 称为 BCD 码. 当一个字节(8 位)中存放一位 BCD 码, 且放在字节的低 4 位, 高 4 位为 0 时称为未组合的 BCD 码. 2. AAA 的调整操作 若 (AL) and 0FH > 9 或 AF=1, 则调整如下: $(AL) \leftarrow (AL) + 6$, $(AH) \leftarrow (AH) + 1$, AF=1, CF ← AF, $(AL) \leftarrow (AL) \text{ and } 0FH$
AAD	未组合十进制数除法调整指令 AAD(ASCII Adjust for Division) 格式: AAD 功能: 在除法指令前对 AX 中的两个未组合十进制数进行调整, 以便能用 DIV 指令实现两个未组合的十进制数的除法运算, 其结果为未组合的十进制数, 商(在 AL 中)和余数(在 AH 中).	说明: 1. AAD 指令是在执行除法 DIV 之前使用的, 以便得到二进制结果存于 AL 中, 然后除以 OPRD, 得到的商在 AL 中, 余数在 AH 中. 2. 示例: MOV BL, 5 MOV AX, 0308H AAD ; $(AL) \leftarrow 1EH + 08H = 26H$, $(AH) \leftarrow 0$ DIV BL ; 商 = 07H → (AL), 余数 = 03H → (AH).
AAM	未组合十进制数乘法调整指令 AAM(ASCII Adjust MULtiply) 格式: AAM 功能: 对两个未组合的十进制数相乘后存于 AX 中的结果进行调整, 产生一个未组合的十进制数存在 AL 中.	说明: 1. 实际上是两个未组合的十进制数字节相乘, 一个 0~9 的数与另一个 0~9 的数相乘其积最大为 81. 为了得到正确的结果, 应进行如下调整: 乘积: $(AH) \leftarrow (AL) / 10$ $(AL) \leftarrow (AL) \text{ MOD } 10$ 2. 本指令应跟在 MUL 指令后使用, 乘积的两位十进制结果, 高位放在 AH 中, 低位放在 AL 中. AH 内容是 MUL 指令的结果被 10 除的商, 即 $(AL) / 10$, 而最后的 AL 内容是乘积被 10 整除的余数(即个位数).
AAS	未组合十进制减法调整指令 AAS(ASCII Adjust for Subtraction) 格式: AAS 功能: 对两个未组合十进制数相减后存于 AL 中的结果进行调整, 调整后产生一个未组合的十进制数且仍存于 AL 中.	说明: 1. 本指令影响标志位 CF 及 AF. 2. 调整操作 若 $(AL) \text{ and } 0FH > 9$ 或 AF=1 则 $(AL) \leftarrow (AL) - 6$, $(AH) \leftarrow (AH) - 1$, CF ← AF, $(AL) \leftarrow (AL) \text{ and } 0FH$, 否则 $(AL) \leftarrow (AL) \text{ and } 0FH$
ADC	带进位加法指令 ADC(Addition Carry)	说明:

	格式: ADC OPRD1, OPRD2 功能: $OPRD1 \leftarrow OPRD1 + OPRD2 + CF$	1. OPRD1 为任一通用寄存器或存储器操作数, 可以是任意一个通用寄存器, 而且还可以是任意一个存储器操作数. OPRD2 为立即数, 也可以是任意一个通用寄存器操作数. 立即数只能用于源操作数. 2. OPRD1 和 OPRD2 均为寄存器是允许的, 一个为寄存器而另一个为存储器也是允许的, 但不允许两个都是存储器操作数. 3. 加法指令运算的结果对 CF、SF、OF、PF、ZF、AF 都会有影响. 以上标志也称为结果标志. 4. 该指令对标志位的影响同 ADD 指令.
ADD	加法指令 ADD(Addition) 格式: ADD OPRD1, OPRD2 功能: 两数相加	说明: 1. OPRD1 为任一通用寄存器或存储器操作数, 可以是任意一个通用寄存器, 而且还可以是任意一个存储器操作数. OPRD2 为立即数, 也可以是任意一个通用寄存器操作数. 立即数只能用于源操作数. 2. OPRD1 和 OPRD2 均为寄存器是允许的, 一个为寄存器而另一个为存储器也是允许的, 但不允许两个都是存储器操作数. 3. 加法指令运算的结果对 CF、SF、OF、PF、ZF、AF 都会有影响. 以上标志也称为结果标志. 加法指令适用于无符号数或有符号数的加法运算.
AND	逻辑与运算指令 AND 格式: AND OPRD1, OPRD2 功能: 对两个操作数实现按位逻辑与运算, 结果送至目的操作数. 本指令可以进行字节或字的‘与’运算, $OPRD1 \leftarrow OPRD1 \text{ and } OPRD2$.	说明: 1. 目的操作数 OPRD1 为任一通用寄存器或存储器操作数. 源操作数 OPRD2 为立即数, 任一通用寄存器或存储器操作数. 2. 示例: AND AL, 0FH ; (AL) $\leftarrow$ (AL) AND 0FH AND AX, BX ; (AX) $\leftarrow$ (AX) AND (BX) AND DX, BUFFER[SI+BX] AND BETA[BX], 00FFH 注意: 两数相与, 有一个数假则值为假
CALL	过程调用指令 CALL 格式: CALL OPRD 功能: 过程调用指令	说明: 1. 其中 OPRD 为过程的目的地址. 2. 过程调用可分为段内调用和段间调

		用两种. 寻址方式也可以分为直接寻址和间接寻址两种. 3. 本指令不影响标志位.
CBW	字节扩展指令 CBW (Convert Byte to Word) 格式: CBW 功能: 将字节扩展为字, 即把 AL 寄存器的符号位扩展到 AH 中.	说明: 1. 两个字节相除时, 先使用本指令形成一个双字节长的被除数. 2. 本指令不影响标志位. 3. 示例: MOV AL, 25 CBW IDIV BYTE PTR DATA1
CLC	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
CLD	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
CLI	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
CMC	处理器控制指令—标志位操作指令	说明: 例如串操作中的程序, 经常用

	格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
CMP	比较指令 CMP (CoMPare) 格式: CMP OPRD1, OPRD2 功能: 对两数进行相减, 进行比较.	说明: 1. OPRD1 为任意通用寄存器或存储器操作数. OPRD2 为任意通用寄存器或存储器操作数, 立即数也可用作源操作数 OPRD2. 2. 对标志位的影响同 SUB 指令, 完成的操作与 SUB 指令类似, 唯一的区别是不将 OPRD1-OPRD2 的结果送回 OPRD1, 而只是比较. 3. 在 8088/8086 指令系统中, 专门提供了一组根据带符号数比较大小后, 实现条件转移的指令.
CMPS	字符串比较指令 格式: CMPS OPRD1, OPRD2 CMPSB CMPSW 功能: 由 SI 寻址的源串中数据与由 DI 寻址的目的串中数据进行比较, 比较结果送标志位, 而不改变操作数本身. 同时 SI, DI 将自动调整.	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址. 2. 本指令影响标志位 AF、CF、OF、SF、PF、ZF. 本指令可用来检查二个字符串是否相同, 可以使用循环控制方法对整串进行比较. 3. 与 MOVSB 相似, CMPS 指令也可以不使用操作数, 此时可用指令 CMPSB 或 CMPSW 分别表示字节串比较或字串比较.
CMPSB	字符串比较指令 格式: CMPS OPRD1, OPRD2 CMPSB CMPSW 功能: 由 SI 寻址的源串中数据与由 DI 寻址的目的串中数据进行比较, 比较结果送标志位, 而不改变操作数本身. 同时 SI, DI 将自动调整.	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址. 2. 本指令影响标志位 AF、CF、OF、SF、PF、ZF. 本指令可用来检查二个字符串是否相同, 可以使用循环控制方法对整串进行比较. 3. 与 MOVSB 相似, CMPS 指令也可以不使用操作数, 此时可用指令 CMPSB 或 CMPSW 分别表示字节串比较或字串比较.

CMPSW	字符串比较指令 格式: CMPS OPRD1, OPRD2 CMPSB CMPSW 功能: 由 SI 寻址的源串中数据与由 DI 寻址的目的串中数据进行比较, 比较结果送标志位, 而不改变操作数本身。 同时 SI, DI 将自动调整。	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址。 2. 本指令影响标志位 AF、CF、OF、SF、PF、ZF。本指令可用来检查二个字符串是否相同, 可以使用循环控制方法对整串进行比较。 3. 与 MOVSB 相似, CMPS 指令也可以不使用操作数, 此时可用指令 CMPSB 或 CMPSW 分别表示字节串比较或字串比较。
CWD	字扩展指令 CWD(Convert Word to Double Word) 格式: CWD 功能: 将字扩展为双字长, 即把 AX 寄存器的符号位扩展到 DX 中。	说明: 1. 两个字或字节相除时, 先用本指令形成一个双字长的被除数。 2. 本指令不影响标志位。 3. 示例: 在 B1、B2、B3 字节类型变量中, 分别存有 8 带符号数 a、b、c, 实现 $(a*b+c)/a$ 运算。
DAA	组合的十进制加法调整指令 DAA(Decimal Adjust for Addition) 格式: DAA 功能: 对 AL 中的两个组合进制数相加的结果进行调整, 调整结果仍放在 AL 中, 进位标志放在 CF 中。	说明: 1. 调整操作如下 (1) 若 (AL) and 0FH > 9 或 AF=1, 则 $(AL) \leftarrow (AL) + 6$, AF ← 1, 对低四位的调整。 (2) 若 (AL) and 0F0H > 90H 或 CF=1, 则 $(AL) \leftarrow (AL) + 60H$, CF ← 1。 2. 示例: (AL)=18H, (BL)=06H ADD AL, BL ; $(AL) \leftarrow (AL) + (BL)$; (AL)=1EH DAA ; (AL)
DAS	组合十进制减法调整指令 DAS(Decimal Adjust for Subtraction) 格式: DAS 功能: 对两个组合十进制数相减后存于 AL 中的结果进行调整, 调整后产生一个组合的十进制数且仍存于 AL 中。	说明: 调整操作 若 (AL) and 0FH > 9 或 AF=1, 则 $(AL) \leftarrow (AL) - 6$, AF=1 若 (AL) and 0F0H > 90H 或 CF=1, 则 $(AL) \leftarrow (AL) - 60H$, CF=1
DEC	减一指令 DEC(Decrement by 1) 格式: DEC OPRD 功能: $OPRD \leftarrow OPRD - 1$	说明: 1. OPRD 为寄存器或存储器操作数。 2. 这条指令执行结果影响 AF、OF、PF、SF、ZF 标志位, 但不影响 CF 标志位。 3. 示例 DEC AX DEC CL DEC WORD PTR[DI] DEC ALFA[DI+BX]
DIV	无符号数除法指令 DIV(DIVision)	说明:

	格式: DIV OPRD 功能: 实现两个无符号二进制数除法运算.	1. 其中 OPRD 为任一个通用寄存器或存储器操作数. 2. 字节相除, 被除数在 AX 中; 字相除, 被除数在 DX, AX 中, 除数在 OPRD 中. 字节除法: $(AL) \leftarrow (AX) / OPRD, (AH) \leftarrow (AX) \text{ MOD } OPRD$ 字除法: $(AX) \leftarrow (DX, AX) / OPRD, (DX) \leftarrow (DX, AX) \text{ MOD } OPRD$
ESC	处理器交权指令 ESC 格式: ESC EXTOPRD, OPRD 功能: 使用本指令可以实现协处理器出放在 ESC 指令代码中的 6 位常数, 该常数指明协处理器要完成的功能. 当源操作数为存储器变量时, 则取出该存储器操作数传送给协处理器.	说明: 1. 其中 EXTOPRD 为外部操作码, OPRD 为源操作数. 2. 本指令不影响标志位.
HLT	处理器暂停指令 HLT 格式: HLT 功能: 使处理器处于暂时停机状态.	说明: 1. 本指令不影响标志位. 2. 由执行 HLT 引起的暂停, 只有 RESET(复位)、NMI(非屏蔽中断请求)、INTR(可屏蔽的外部中断请求)信号可以使 其退出暂停状态. 它可用于等待中断的到来或多机系统的同步操作.
IDIV	带符号数除法指令 IDIV(Integer DIVision) 格式: IDIV OPRD 功能: 这实现两个带符号数的二进制除法运算.	说明: 1. 其中 OPRD 为任一通用寄存器或存储器操作数. 2. 理由与 IMUL 相同, 只有 IDIV 指令, 才能得到符号数相除的正确结果. 3. 当被除数为 8 位, 在进行字节除法前, 应把 AL 的符号位扩充至 AH 中. 在 16 位除法时, 若被除数为 16 位, 则应将 AX 中的符号位扩到 DX 中.
IMUL	带符号数乘法指令 IMUL(Integer MULtiply) 格式: IMUL OPRD 功能: 完成两个带符号数的相乘	说明: 1. 其中 OPRD 为任一通用寄存器或存储器操作数. 2. MUL 指令对带符号相乘时, 不能得到正确的结果. 例如: (AL)=255 (CL)=255 MUL CL (AX)=65025 注意: 这对无符号数讲, 结果是正确

		的,但对带符号数讲,相当于 $(-1)*(-1)$ 结果应为+1,而 65025 对应的带符号数为-511,显然是不正确的.
IN	输入指令 IN 格式: IN AL, n ; (AL) ← (n) IN AX, n ; (AX) ← (n+1), (n) IN AL, DX ; (AL) ← [(DX)] IN AX, DX ; (AX) ← [(DX)+1], [(DX)] 功能: 输入指令	说明: 1. 其中 n 为 8 位的端口地址,当字节输入时,将端口地址 n+1 的内容送至 AH 中,端口地址 n 的内容送 AL 中. 2. 端口地址也可以是 16 位的,但必须将 16 位的端口地址送入 DX 中.当字节寻址时,由 DX 内容作端口地址的内容送至 AL 中; 当输入数据字时, [(DX)+1] 送 AH, [(DX)] 送 AL 中,用符号: (AX) ← [(DX)+1], [(DX)] 表示.
INC	加 1 指令 INC (INCRement by 1) 格式: INC OPRD 功能: OPRD ← OPRD+1	说明: 1. OPRD 为寄存器或存储器操作数. 2. 这条指令执行结果影响 AF、OF、PF、SF、ZF 标志位,但不影响 CF 标志位. 3. 示例: INC SI; (SI) ← (SI)+1 INC WORD PTR[BX] INC BYTE PTR[BX+DI] INC CL; (CL) ← (CL)+1 注意: 上述第二,三两条指令,是对存储字及存储字节的内容加 1 以替代原来的内容.
INT	软中断指令 INT 格式: INT n 其中 n 为软中断的类型号. 功能: 本指令将产生一个软中断,把控制转向一个类型号为 n 的软中断,该中断处理程序入口地址在中断向量表的 n*4 地址 处的二个存储器字(4 个单元)中.	说明: 操作过程与 INTO 指令雷同,只需将 10H 改为 n*4 即可.所以,本指令也将影响标志位 IF 及 TF.
INTO	溢出中断指令 INTO (INTerrupt if Overflow) 格式: INTO 功能: 本指令检测 OF 标志位,当 OF=1 时,说明已发生溢出,立即产生一个中断类型 4 的中断,当 OF=0 时,本指令不起作用.	说明: 1. 本指令影响标志位 IF 及 TF. 2. 本指令可用于溢出处理,当 OF=1 时,产生一个类型 4 的软中断.在中断处理程序中完成溢出的处理操作.
IRET	中断返回指令 IRET 格式: IRET 功能: 用于中断处理程序中,从中断程序的断点处返回,继续执行原程序.	说明: 1. 本指令将影响所有标志位. 2. 无论是软中断,还是硬中断,本指令均可使其返回到中断程序的断点处继

		续执行原程序.
JA	条件转移指令 JA/JNBE 格式: JA/JNBE 标号 功能: 为高于/不低于等于的转移指令	说明: 1. 例如两个符号数 a, b 比较时, $a > b$ (即 $CF=0, ZF=0$) 时转移. 因为单一标志位 $CF=0$, 只表示 $a \geq b$. 2. JA/JNBE 是同一条指令的两种不同的助记符. 3. 该指令用于无符号数进行条件转移
JAE	条件转移指令 JAE/JNB 格式: JAE/JNB 标号 功能: 为高于等于/不低于的转移指令	说明: 1. JAE/JNB 是同一条指令的两种不同的助记符. 2. 该指令用于无符号数进行条件转移.
JB	条件转移指令 JB/JNAE 格式: JB/JNAE 标号 功能: 低于/不高于等于时转移	说明: 该指令用于无符号数的条件转移
JBE	条件转移指令 JBE/JNA 格式: JBE/JNA 标号 功能: 低于等于/不高于时转移	说明: 该指令用于无符号数的条件转移
JC	条件转移指令 JC 格式: JC 标号 功能: $CF=1$, 转至标号处执行	说明: JC 为根据标志位 CF 进行转移的指令
JE	条件转移指令 JE/JZ 格式: JE/JZ 标号 功能: $ZF=1$, 转至标号处执	说明: 1. 指令 JE 与 JZ 等价, 它们是根据标志位 ZF 进行转移的指令 2. JE, JZ 均为一条指令的两种助记符表示方法
JG	条件转移指令 JG/JNLE 格式: JG/JNLE 标号 功能: 大于/不小于等于时转移	说明: 用于带符号数的条件转移指令
JGE	条件转移指令 JGE/JNL 格式: JGE/JNL 标号 功能: 大于等于/不小于时转移	说明: 用于带符号数的条件转移指令
JL	条件转移指令 JL/JNGE 格式: JL/JNGE 标号 功能: 小于/不大于等于时转移	说明: 用于带符号数的条件转移指令
JLE	条件转移指令 JLE/JNG 格式: JLE/JNG 标号 功能: 小于等于/不大于时转移	说明: 用于带符号数的条件转移指令
JMP	无条件转移指令 JMP 格式: JMP OPRD 功能: JMP 指令将无条件地控制程序转移到目的地址去执行. 当目的地址仍在同一个代码段内, 称为段内转移; 当目标	说明: 1. 其中 OPRD 为转移的目的地址. 程序转移到目的地址所指向的指令继续往下执行. 2. 本组指令对标志位无影响.

	地址不在同一个代码段内, 则称为段间转移. 这两种情况都将产生不同的指令代码, 以便能正确地生成目的地址, 在段内转移时, 指令只要能提供目的地址的段内偏移量即够了; 而在段间转移时, 指令应能提供目的地址的段地址及段内偏移地址值.	3. <1> 段内直接转移指令: JMP NEAR 标号 <2> 段内间接转移指令: JMP OPRD <3> 段间直接转移指令: JMP FAR 标号 <4> 段间间接转移指令: JMP OPRD 其中的 OPRD 为存储器双字操作数. 段间间接转移只能通过存储器操作数来实现.
JNA	条件转移指令 JBE/JNA 格式: JBE/JNA 标号 功能: 低于等于/不高于时转移	说明: 该指令用于无符号数的条件转移
JNAE	条件转移指令 JB/JNAE 格式: JB/JNAE 标号 功能: 低于/不高于等于时转移	说明: 该指令用于无符号数的条件转移
JNB	条件转移指令 JAE/JNB 格式: JAE/JNB 标号 功能: 为高于等于/不低于的转移指令	说明: 1. JAE/JNB 是同一条指令的两种不同的助记符. 2. 该指令用于无符号数进行条件转移.
JNBE	条件转移指令 JA/JNBE 格式: JA/JNBE 标号 功能: 为高于/不低于等于的转移指令	说明: 1. 例如两个符号数 a, b 比较时, $a > b$ (即 $CF=0, ZF=0$) 时转移. 因为单一标志位 $CF=0$, 只表示 $a \geq b$. 2. JA/JNBE 是同一条指令的两种不同的助记符. 3. 该指令用于无符号数进行条件转移
JNC	条件转移指令 JNC 格式: JNC 标号 功能: $CF=0$, 转至标号处执行	说明: JNC 为根据标志位 CF 进行转移的指令
JNE	条件转移指令 JNE/JNZ 格式: JNE/JNZ 标号 功能: $ZF=0$, 转至标号处执行	说明: 1. 指令 JNE 与 JNZ 等价, 它们是根据标志位 ZF 进行转移的指令 2. JNE, JNZ 均为一条指令的两种助记符表示方法
JNG	条件转移指令 JLE/JNG 格式: JLE/JNG 标号 功能: 小于等于/不大于时转移	说明: 用于带符号数的条件转移指令
JNGE	条件转移指令 JL/JNGE 格式: JL/JNGE 标号 功能: 小于/不大于等于时转移	说明: 用于带符号数的条件转移指令
JNL	条件转移指令 JGE/JNL 格式: JGE/JNL 标号 功能: 大于等于/不小于时转移	说明: 用于带符号数的条件转移指令
JNLE	条件转移指令 JG/JNLE 格式: JG/JNLE 标号	说明: 用于带符号数的条件转移指令

	功能：大于/不小于等于时转移	
JNO	条件转移指令 JNO 格式：JNO 标号 功能：OF=0, 转至标号处执行	说明：JNO 是根据溢出标志位 OF 进行转移的指令
JNP	条件转移指令 JNP/JPO 格式：JNP/JPO 标号 功能：PF=0, 转至标号处执行	说明： 1. 指令 JNP 与 JPO, 它们是根据奇偶标志位 PF 进行转移的指令 2. JNP, JPO 均为一条指令的两种助记符表示方法
JNS	条件转移指令 JNS 格式：JNS 标号 功能：SF=0, 转至标号处执行	说明：JNS 是根据符号标志位 SF 进行转移的指令
JNZ	条件转移指令 JNE/JNZ 格式：JNE/JNZ 标号 功能：ZF=0, 转至标号处执行	说明： 1. 指令 JNE 与 JNZ 等价, 它们是根据标志位 ZF 进行转移的指令 2. JNE, JNZ 均为一条指令的两种助记符表示方法
JO	条件转移指令 JO 格式：JO 标号 功能：OF=1, 转至标号处执行	说明：JO 是根据溢出标志位 OF 进行转移的指令
JP	条件转移指令 JP/JPE 格式：JP/JPE 标号 功能：PF=1, 转至标号处执行	说明： 1. 指令 JP 与 JPE, 它们是根据奇偶标志位 PF 进行转移的指令 2. JP, JPE 均为一条指令的两种助记符表示方法
JPE	条件转移指令 JP/JPE 格式：JP/JPE 标号 功能：PF=1, 转至标号处执行	说明： 1. 指令 JP 与 JPE, 它们是根据奇偶标志位 PF 进行转移的指令 2. JP, JPE 均为一条指令的两种助记符表示方法
JPO	条件转移指令 JNP/JPO 格式：JNP/JPO 标号 功能：PF=0, 转至标号处执行	说明： 1. 指令 JNP 与 JPO, 它们是根据奇偶标志位 PF 进行转移的指令 2. JNP, JPO 均为一条指令的两种助记符表示方法
JS	条件转移指令 JS 格式：JS 标号 功能：SF=1, 转至标号处执行	说明：JS 是根据符号标志位 SF 进行转移的指令
JZ	条件转移指令 JE/JZ 格式：JE/JZ 标号 功能：ZF=1, 转至标号处执行	说明： 1. 指令 JE 与 JZ 等价, 它们是根据标志位 ZF 进行转移的指令 2. JE, JZ 均为一条指令的两种助记符表示方法
LAHF	标志传送指令 LAHF	说明：该指令不影响 FLAG 的原来内

	格式: LAHF 功能: 取 FLAG 标志寄存器低 8 位至 AH 寄存器. $(AH) \leftarrow (FLAG)_{7 \sim 0}$	容, AH 只是复制了原 FLAG 的低 8 位内容.
LDS	从存储器取出 32 位地址的指令 LDS 格式: LDS OPRD1, OPRD2 功能: 从存储器取出 32 位地址的指令.	说明: OPRD1 为任意一个 16 位的寄存器. OPRD2 为 32 位的存储器地址. 示例: LDS SI, ABCD LDS BX, FAST[SI] LDS DI, [BX] 注意: 上面 LDS DI, [BX] 指令的功能是把 BX 所指的 32 位地址指针的段地址送入 DS, 偏移地址送入 DI.
LEA	有效地址传送指令 LEA 格式: LEA OPRD1, OPRD2 功能: 将源操作数给出的有效地址传送到指定的寄存器中.	说明: 1. OPRD1 为目的操作数, 可为任意一个 16 位的通用寄存器. OPRD2 为源操作数, 可为变量名、标号或地址表达式. 示例: LEA BX, DATA1 LEA DX, BETA[BX+SI] LEA BX BX, [BP], [DI] 2. 本指令对标志位无影响.
LES	从存储器取出 32 位地址的指令 LES 格式: LES OPRD1, OPRD2 功能: 从存储器取出 32 位地址的指令.	说明: OPRD1 为任意一个 16 位的寄存器. OPRD2 为 32 位的存储器地址. 示例: LES SI, ABCD LES BX, FAST[SI] LES DI, [BX] 注意: 上面 LES DI, [BX] 指令的功能是把 BX 所指的 32 位地址指针的段地址送入 ES, 偏移地址送入 DI.
LOCK	封锁总线指令 LOCK 格式: LOCK 功能: 指令是一个前缀, 可放在指令的前面, 告诉 CPU 在执行该指令时, 不允许其它设备对总线进行访问.	无可用的信息! 用户可自行添加!
LODS	取字符串元素指令 LODS 格式: LODS OPRD 其中 OPRD 为源字符串符号地址. 功能: 把 SI 寻址的源串的数据字节送 AL 或数据字送 AX 中去, 并根据 DF 的值修改地址指针 SI 进行自动调整.	说明: 1. 本指令不影响标志位. 2. 当不使用操作数时, 可用 LODS(字节串)或 LODSW(字串)指令.
LOOP	循环控制指令 LOOP 格式: LOOP 标号 功能: $(CX) \leftarrow (CX) - 1$, $(CX) \neq 0$, 则转移	说明: 1. 本指令是用 CX 寄存器作为计数器, 来控制程序的循环.

	至标号处循环执行, 直至 (CX)=0, 继续执行后继指令.	2. 它属于段内 SHORT 短类型转移, 目的地址必须距本指令在-128 到+127 个字节的范围内.
LOOPE	循环控制指令 LOOPZ/LOOPE 格式: LOOPZ/LOOPE 标号 功能: (CX)←(CX)-1, (CX)≠0 且 ZF=1 时, 转至标号处循环	说明: 1. 本指令是用 CX 寄存器作为计数器, 来控制程序的循环. 2. 它属于段内 SHORT 短类型转移, 目的地址必须距本指令在-128 到+127 个字节的范围内. 3. 以上两种助记符等价.
LOOPNE	循环控制指令 LOOPNZ/LOOPNE 格式: LOOPNZ/LOOPNE 标号 功能: (CX)←(CX)-1, (CX)≠0 且 ZF=0 时, 转至标号处循环	说明: 1. 本指令是用 CX 寄存器作为计数器, 来控制程序的循环. 2. 它属于段内 SHORT 短类型转移, 目的地址必须距本指令在-128 到+127 个字节的范围内. 3. 以上两种助记符等价.
LOOPNZ	循环控制指令 LOOPNZ/LOOPNE 格式: LOOPNZ/LOOPNE 标号 功能: (CX)←(CX)-1, (CX)≠0 且 ZF=0 时, 转至标号处循环	说明: 1. 本指令是用 CX 寄存器作为计数器, 来控制程序的循环. 2. 它属于段内 SHORT 短类型转移, 目的地址必须距本指令在-128 到+127 个字节的范围内. 3. 以上两种助记符等价.
LOOPZ	循环控制指令 LOOPZ/LOOPE 格式: LOOPZ/LOOPE 标号 功能: (CX)←(CX)-1, (CX)≠0 且 ZF=1 时, 转至标号处循环	说明: 1. 本指令是用 CX 寄存器作为计数器, 来控制程序的循环. 2. 它属于段内 SHORT 短类型转移, 目的地址必须距本指令在-128 到+127 个字节的范围内. 3. 以上两种助记符等价.
MOVE	数据传送指令 MOV 格式: MOV OPRD1, OPRD2 功能: 本指令将一个源操作数送到目的操作数中, 即 OPRD1←OPRD2.	说明: 1. OPRD1 为目的操作数, 可以是寄存器、存储器、累加器. OPRD2 为源操作数, 可以是寄存器、存储器、累加器和立即数. 2. MOV 指令以分为以下四种情况: 〈1〉寄存器与寄存器之间的数据传送指令 〈2〉立即数到通用寄存器数据传送指令 〈3〉寄存器与存储器之间的数据传送指令 〈4〉立即数到存储器的数据传送

		3. 本指令不影响状态标志位
MOVS	字符串传送指令 MOVS 格式: MOVS OPRD1, OPRD2 MOVSB MOVSW 功能: OPRD1←—OPRD2.	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址. 2. 字节串操作: 若 DF=0, 则作加, 若 DF=1, 则作减. 3. 对字串操作时: 若 DF=0, 则作加, 若 DF=1, 则作减, . 4. 在指令中不出现操作数时, 字节串传送格式为 MOVSB、字串传送格式为 MOVSW. 5. 本指令不影响标志位.
MOVSB	字符串传送指令 MOVS 格式: MOVS OPRD1, OPRD2 MOVSB MOVSW 功能: OPRD1←—OPRD2.	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址. 2. 字节串操作: 若 DF=0, 则作加, 若 DF=1, 则作减. 3. 对字串操作时: 若 DF=0, 则作加, 若 DF=1, 则作减, . 4. 在指令中不出现操作数时, 字节串传送格式为 MOVSB、字串传送格式为 MOVSW. 5. 本指令不影响标志位.
MOVSW	字符串传送指令 MOVS 格式: MOVS OPRD1, OPRD2 MOVSB MOVSW 功能: OPRD1←—OPRD2.	说明: 1. 其中 OPRD2 为源串符号地址, OPRD1 为目的串符号地址. 2. 字节串操作: 若 DF=0, 则作加, 若 DF=1, 则作减. 3. 对字串操作时: 若 DF=0, 则作加, 若 DF=1, 则作减, . 4. 在指令中不出现操作数时, 字节串传送格式为 MOVSB、字串传送格式为 MOVSW. 5. 本指令不影响标志位.
MUL	无符号数乘法指令 MUL (MULtiply) 格式: MUL OPRD 功能: 乘法操作.	说明: 1. OPRD 为通用寄存器或存储器操作数. 2. OPRD 为源操作数, 即作乘数. 目的操作数是隐含的, 即被乘数总是指定为累加器 AX 或 AL 的内容. 3. 16 位乘法时, AX 中为被乘数. 8 位乘法时, AL 为被乘数. 当 16 位乘法时, 32 位的乘积存于 DX 及 AX 中; 8 位乘法的 16 位乘积存于 AX 中.

		4. 操作过程: 字节相乘: $(AX) \leftarrow (AL) * OPRD$, 当结果的高位字节(AH)不等于0时, 则 $CF=1$ 、 $OF=1$.
NEG	取补指令 NEG (NEGate) 格式: NEG OPRD 功能: 对操作数 OPRD 进行取补操作, 然后将结果送回 OPRD. 取补操作也叫作求补操作, 就是求一个数的相反数的补码.	说明: 1. OPRD 为任意通用寄存器或存储器操作数. 2. 示例: $(AL)=44H$, 取补后, $(AL)=0BCH(-44H)$. 3. 本指令影响标志位 CF、OF、SF、PF、ZF 及 AF.
NOP	空操作指令 NOP 格式: NOP 功能: 本指令不产生任何结果, 仅消耗几个时钟周期的时间, 接着执行后续指令, 常用于程序的延时等.	说明: 本指令不影响标志位.
NOT	逻辑非运算指令 NOT 格式: NOT OPRD 功能: 完成对操作数按位求反运算(即 0 变 1, 1 变 0), 结果送回原操作数.	说明: 1. 其中 OPRD 可为任一通用寄存器或存储器操作数. 2. 本指令可以进行字或字节‘非’运算. 3. 本指令不影响标志位.
OR	逻辑或指令 OR 格式: OR OPRD1, OPRD2 功能: OR 指令完成对两个操作数按位的‘或’运算, 结果送至目的操作数中, 本指令可以进行字节或字的‘或’运算. $OPRD1 \leftarrow OPRD1 OR OPRD2$.	说明: 1. 其中 OPRD1, OPRD2 含义与 AND 指令相同, 对标志位的影响也与 AND 指令相同. 2. 两数相或, 有一个数为真则值为真.
OUT	输出指令 OUT 格式: OUT n, AL ; $(n) \leftarrow (AL)$ 功能: 输出指令	说明: 1. $OUT\ n, AX ; (n+1), (n) \leftarrow (AX)$ $OUT\ DX, AL ; [(DX)] \leftarrow (AL)$ $OUT\ DX, AX ; [(DX)+1], [(DX)] \leftarrow (AX)$ 2. 输入指令及输出指令对标志位都不影响.
POP	堆栈操作指令 PUSH 和 POP 格式: PUSH OPRD POP OPRD 功能: 实现压入操作的指令是 PUSH 指令; 实现弹出操作的指令是 POP 指令.	说明: 1. OPRD 为 16 位(字)操作数, 可以是寄存器或存储器操作数. 2. POP 指令的操作过程是: POP OPRD: $OPRD \leftarrow ((SP))$, $(SP) \leftarrow (SP) + 2$ 它与压入操作相反, 是先弹出栈顶的数项, 然后再修改指针 SP 的内容. 3. 示例: POP AX POP DS POP DATA1 POP ALFA[BX][DI]

		4. PUSH 和 POP 指令对状态标志位没有影响.
POPF	标志传送指令 POPF 格式: POPF 功能: 本指令的功能与 PUSHF 相反, 在子程序调用和中断服务程序中, 往往用 PUSHF 指令保护 FLAG 的内容, 用 POPF 指令将保护的 FLAG 内容恢复.	说明: 如果对堆栈中的原 FLAG 内容进行修改, 如对 TF 等标志位进行修改, 然后再弹回标志位寄存器 FLAG. 这是通过指令修改 TF 标志的唯一方法.
PUSH	堆栈操作指令 PUSH 和 POP 格式: PUSH OPRD POP OPRD 功能: 实现压入操作的指令是 PUSH 指令; 实现弹出操作的指令是 POP 指令.	说明: 1. OPRD 为 16 位(字)操作数, 可以是寄存器或存储器操作数. 2. PUSH 的操作过程是: $(SP) \leftarrow (SP) - 2$, $((sp)) \leftarrow OPRD$ 即先修改堆栈指针 SP (压入时为自动减 2), 然后, 将指定的操作数送入新的栈顶位置. 此处的 $((SP)) \leftarrow OPRD$, 也可以理解为: $[(SS) * 16 + (SP)] \leftarrow OPRD$ 或 $[SS:SP] \leftarrow OPRD$
PUSHF	标志传送指令 PUSHF 格式: PUSHF 功能: 本指令可以把标志寄存器的内容保存到堆栈中去	
RCL	循环移位指令 格式: ROL OPRD1, COUNT ; 不含进位标志位 CF 在循环中的左循环移位指令. ROR OPRD1, COUNT ; 不含进位标志位 CF 在循环中的右循环移位指令. RCL OPRD1, COUNT ; 带进位的左循环移位指令. RCR OPRD1, COUNT ; 带进位的右循环移位指令.	说明: 1. 本指令组只影响标志 CF、OF. OF 由移入 CF 的内容决定, OF 取决于移位一次后符号位是否改变, 如改变, 则 OF=1. 2. 由于是循环移位, 所以对字节移位 8 次; 对字移位 16 次, 就可恢复为原操作数. 由于带 CF 的循环移位, 可以将 CF 的内容移入, 所以可以利用它实现多字节的循环.
RCR	循环移位指令 格式: ROL OPRD1, COUNT ; 不含进位标志位 CF 在循环中的左循环移位指令. ROR OPRD1, COUNT ; 不含进位标志位 CF 在循环中的右循环移位指令. RCL OPRD1, COUNT ; 带进位的左循环移位指令. RCR OPRD1, COUNT ; 带进位的右循环移位指令.	说明: 1. 本指令组只影响标志 CF、OF. OF 由移入 CF 的内容决定, OF 取决于移位一次后符号位是否改变, 如改变, 则 OF=1. 2. 由于是循环移位, 所以对字节移位 8 次; 对字移位 16 次, 就可恢复为原操作数. 由于带 CF 的循环移位, 可以将 CF 的内容移入, 所以可以利用它实现多字节的循环. 注意: 以上程序中的指令 SHR AL, CL 如改为 SAR AL, CL, 虽然最高 4 位可移入低 4 位, 但最高位不为 0, 故应加入一条

		指令 AND AL, 0FH. 否则, 若最高位不为 0 时, 将得到错误结果.
REP	重复前缀的说明 格式: REP ;CX<>0 重复执行字符串指令 REPZ/REPE ;CX<>0 且 ZF=1 重复执行字符串指令 REPZ/REPNE ;CX<>0 且 ZF=0 重复执行字符串指令 功能: 在串操作指令前加上重复前缀, 可以对字符串进重复处理. 由于加上重复前缀后, 对应的指令代码是不同的, 所以指令的功能便具有重复处理的功能, 重复的次数存放在 CX 寄存器中.	说明: 1. REP 与 MOVS 或 STOS 串操作指令相结合使用, 完成一组字符的传送或建立一组相同数据的字符串. 2. REPZ/REPE 常用与 CMPS 串操作指令结合使用, 可以完成两组字符串的比较. 3. REPZ/REPE 常与 SCAS 指令结合使用, 可以完成在一个字符串中搜索一个关键字. 4. REPZ/REPNE 与 CMPS 指令结合使用, 表示当串未结束 (CX=1) 且当对应串元素不相同 (ZF=0) 时, 继续重复执行串比较指令.
REPE	重复前缀的说明 格式: REP ;CX<>0 重复执行字符串指令 REPZ/REPE ;CX<>0 且 ZF=1 重复执行字符串指令 REPZ/REPNE ;CX<>0 且 ZF=0 重复执行字符串指令 功能: 在串操作指令前加上重复前缀, 可以对字符串进重复处理. 由于加上重复前缀后, 对应的指令代码是不同的, 所以指令的功能便具有重复处理的功能, 重复的次数存放在 CX 寄存器中.	说明: 1. REPZ/REPE 常用与 CMPS 串操作指令结合使用, 可以完成两组字符串的比较. 2. REPZ/REPE 常与 SCAS 指令结合使用, 可以完成在一个字符串中搜索一个关键字. 3. REPZ/REPNE 与 CMPS 指令结合使用, 表示当串未结束 (CX=1) 且当对应串元素不相同 (ZF=0) 时, 继续重复执行串比较指令. 4. REPZ/REPNE 与 SCAS 指令结合使用, 表示串未结束 (CX=1) 且当关键字与串元素不相同 (ZF=0) 时, 继续重复执行串搜索指令.
REPNE	重复前缀的说明 格式: REP ;CX<>0 重复执行字符串指令 REPZ/REPE ;CX<>0 且 ZF=1 重复执行字符串指令 REPZ/REPNE ;CX<>0 且 ZF=0 重复执行字符串指令	说明: 1. REPZ/REPE 常用与 CMPS 串操作指令结合使用, 可以完成两组字符串的比较. 2. REPZ/REPE 常与 SCAS 指令结合使用, 可以完成在一个字符串中搜索一个关键字. 3. REPZ/REPNE 与 CMPS 指令结合使用, 表示当串未结束 (CX=1) 且当对应串元素不相同 (ZF=0) 时, 继续重复执行串比较指令. 4. REPZ/REPNE 与 SCAS 指令结合使用, 表示串未结束 (CX=1) 且当关键字与串

		元素不相同 (ZF=0) 时, 继续重复执行串搜索指令.
REPZ	重复前缀的说明 格式: REP ;CX<>0 重复执行字符串指令 REPZ/REPE ;CX<>0 且 ZF=1 重复执行字符串指令 REPZ/REPNE ;CX<>0 且 ZF=0 重复执行字符串指令	说明: 1. REPZ/REPE 常用与 CMPS 串操作指令结合使用, 可以完成两组字符串的比较. 2. REPZ/REPE 常与 SCAS 指令结合使用, 可以完成在一个字符串中搜索一个关键字. 3. REPZ/REPNE 与 CMPS 指令结合使用, 表示当串未结束 (CX=1) 且当对应串元素不相同 (ZF=0) 时, 继续重复执行串比较指令. 4. REPZ/REPNE 与 SCAS 指令结合使用, 表示串未结束 (CX=1) 且当关键字与串元素不相同 (ZF=0) 时, 继续重复执行串搜索指令.
REPZ	重复前缀的说明 格式: REP ;CX<>0 重复执行字符串指令 REPZ/REPE ;CX<>0 且 ZF=1 重复执行字符串指令 REPZ/REPNE ;CX<>0 且 ZF=0 重复执行字符串指令 功能: 在串操作指令前加上重复前缀, 可以对字符串进重复处理. 由于加上重复前缀后, 对应的指令代码是不同的, 所以指令的功能便具有重复处理的功能, 重复的次数存放在 CX 寄存器中.	说明: 1. REPZ/REPE 常用与 CMPS 串操作指令结合使用, 可以完成两组字符串的比较. 2. REPZ/REPE 常与 SCAS 指令结合使用, 可以完成在一个字符串中搜索一个关键字. 3. REPZ/REPNE 与 CMPS 指令结合使用, 表示当串未结束 (CX=1) 且当对应串元素不相同 (ZF=0) 时, 继续重复执行串比较指令. 4. REPZ/REPNE 与 SCAS 指令结合使用, 表示串未结束 (CX=1) 且当关键字与串元素不相同 (ZF=0) 时, 继续重复执行串搜索指令.
RET	返回指令 RET 格式: RET 功能: 当调用的过程结束后实现从过程返回至原调用程序的下一条指令, 本指令不影响标志位.	说明: 由于在过程定义时, 已指明其近 (NEAR) 或远 (FAR) 的属性, 所以 RET 指令根据段内调用与段间调用, 执行不同的操作 对段内调用: 返回时, 由堆栈弹出一个字的返回地址的段内偏移量至 IP. 对段外调用: 返回时, 由堆栈弹出的第一个字为返回地址的段内偏移量, 将其送入 IP 中, 由堆栈弹出第二个字为返回地址的段基址, 将其送入 CS 中.
ROL	循环移位指令 格式: ROL OPRD1, COUNT ;不含进位标志	说明: 1. 本指令组只影响标志 CF、OF. OF 由

	位 CF 在循环中的左循环移位指令. ROR OPRD1, COUNT ;不含进位标志位 CF 在循环中的右循环移位指令. RCL OPRD1, COUNT ;带进位的左循环移位 指令. RCR OPRD1, COUNT ;带进位的右循环移位 指令.	移入 CF 的内容决定, OF 取决于移位一 次后符号位是否改变, 如改变, 则 OF=1. 2. 由于是循环移位, 所以对字节移位 8 次; 对字移位 16 次, 就可恢复为原操 作数. 由于带 CF 的循环移位, 可以将 CF 的内容移入, 所以可以利用它实现多字节的循环.
ROR	循环移位指令 格式: ROL OPRD1, COUNT ;不含进位标志位 CF 在循环中的左循环移位指令. ROR OPRD1, COUNT ;不含进位标志位 CF 在循环中的右循环移位指令. RCL OPRD1, COUNT ;带进位的左循环移位 指令. RCR OPRD1, COUNT ;带进位的右循环移位 指令.	说明: 1. 本指令组只影响标志 CF、OF. OF 由 移入 CF 的内容决定, OF 取决于移位一 次后符号位是否改变, 如改变, 则 OF=1. 2. 由于循环移位, 所以对字节移位 8 次; 对字移位 16 次, 可恢复为原操作 数.
SAHF	标志传送指令 SAHF 格式: SAHF 功能: 将 AH 存至 FLAG 低 8 位	说明: 本指令将用 AH 的内容改写 FLAG 标志寄存器中的 SF、ZF、AF、PF、和 CF 标志, 从而改变原来的标志位.
SAL	算术左移指令 SAL (Shift Arithmetic Left) 格式: SAL OPRD1, COUNT 功能: 其中 OPRD1, COUNT 与指令 SHL 相 同. 本指令与 SHL 的功能也完全相同, 这 是因为逻辑左移指令与算术左移指令所 要完成的操作是一样的.	说明: 1. 其中 OPRD1 为目的操作数, 可以是 通用寄存器或存储器操作数. 2. COUNT 代表移位的次数(或位数). 移 位一次, COUNT=1; 移位多于 1 次 时, COUNT=(CL), (CL) 中为移位的次数.
SAR	算术右移指令 SAR 格式: SAR OPRD1, COUNT 功能: 本指令通常用于对带符号数减半 的运算中, 因而在每次右移时, 保持最高 位(符号位)不变, 最低位右移至 CF 中.	说明: 1. 其中 OPRD1 为目的操作数, 可以是 通用寄存器或存储器操作数. 2. COUNT 代表移位的次数(或位数). 移 位一次, COUNT=1; 移位多于 1 次 时, COUNT=(CL), (CL) 中为移位的次数.
SBB	带借位减去指令 SBB (SuBtraction with Borrow) 格式: SBB OPRD1, OPRD2 功能: 是进行两个操作数的相减再减去 CF 进位标志位, 即从 $OPRD1 \leftarrow OPRD1 - OPRD2 - CF$, 其结果放在 OPRD1 中.	说明: 示例 SBB DX, CX SBB AX, DATA1 SBB BX, 2000H SBB ALFA[BX+SI], SI SBB BETAP[DI, 030AH
SCAS	字符串搜索指令 SCAS 格式: SCAS OPRD SCASB SCASW	说明: 1. 其中 OPRD 为目的串符号地址. 2. 本指令影响标志 AF、CF、OF、PF、 SF、ZF. 该指令可查找字符串中的一个

	功能：把 AL(字节串)或 AX(字串)的内容与由 DI 寄存器寻址的目的串中的数据相减, 结果置标志位, 但不改变任一操作数本身. 地址指针 DI 自动调整.	关键字, 只需在本指令执行前, 把关键字放在 AL(字节)或 AX(字串)中, 用重复前缀可在整串中查找. 指令中不使用操作数时, 可用指令格式 SCASB, SCASW, 分别表示字节串或字串搜索指令.
SCASB	字符串搜索指令 SCAS 格式: SCAS OPRD SCASB SCASW 功能：把 AL(字节串)或 AX(字串)的内容与由 DI 寄存器寻址的目的串中的数据相减, 结果置标志位, 但不改变任一操作数本身. 地址指针 DI 自动调整.	说明: 1. 其中 OPRD 为目的串符号地址. 2. 本指令影响标志 AF、CF、OF、PF、SF、ZF. 该指令可查找字符串中的一个关键字, 只需在本指令执行前, 把关键字放在 AL(字节)或 AX(字串)中, 用重复前缀可在整串中查找. 指令中不使用操作数时, 可用指令格式 SCASB, SCASW, 分别表示字节串或字串搜索指令.
SCASW	字符串搜索指令 SCAS 格式: SCAS OPRD SCASB SCASW 功能：把 AL(字节串)或 AX(字串)的内容与由 DI 寄存器寻址的目的串中的数据相减, 结果置标志位, 但不改变任一操作数本身. 地址指针 DI 自动调整.	说明: 1. 其中 OPRD 为目的串符号地址. 2. 本指令影响标志 AF、CF、OF、PF、SF、ZF. 该指令可查找字符串中的一个关键字, 只需在本指令执行前, 把关键字放在 AL(字节)或 AX(字串)中, 用重复前缀可在整串中查找. 指令中不使用操作数时, 可用指令格式 SCASB, SCASW, 分别表示字节串或字串搜索指令.
SHL	逻辑左移指令 SHL(Shift logical left) 格式: SHL OPRD1, COUNT 功能：对给定的目的操作数左移 COUNT 次, 每次移位时最高位移入标志位 CF 中, 最低位补零.	说明: 1. 其中 OPRD1 为目的操作数, 可以是通用寄存器或存储器操作数. 2. COUNT 代表移位的次数(或位数). 移位一次, COUNT=1; 移位多于 1 次时, COUNT=(CL), (CL) 中为移位的次数. 3. 例如: SHL AL, 1 SHL CX, 1 SHL ALFA[DI] 或者: MOV CL, 3 SHL DX, CL SHL ALFA[DI], CL
SHR	逻辑右移指令 SHR 格式: SHR OPRD1, COUNT 功能：本指令实现由 COUNT 决定次数的逻辑右移操作, 每次移位时, 最高位补 0, 最低位移至标志位 CF 中.	说明: 1. 其中 OPRD1 为目的操作数, 可以是通用寄存器或存储器操作数. 2. COUNT 代表移位的次数(或位数). 移位一次, COUNT=1; 移位多于 1 次时, COUNT=(CL), (CL) 中为移位的次数.

		3. 影响标志位 OF, PF, SF, ZF, CF.
STC	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
STD	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
STI	处理器控制指令—标志位操作指令 格式: CLC ;置 CF=0 STC ;置 CF=1 CMC ;置 CF=(Not CF) 进位标志求反 CLD ;置 DF=0 STD ;置 DF=1 CLI ;置 IF=0, CPU 禁止响应外部中断 STI ;置 IF=1, 使 CPU 允许响应外部中断 功能: 完成对标志位的置位、复位等操作.	说明: 例如串操作中的程序, 经常用 CLD 指令清方向标志使 DF=0, 在串操作指令执行时, 按增量的方式修改指针.
STOS	字符串存储指令 STOS 格式: STOS OPRD 功能: 把 AL(字节)或 AX(字)中的数据存储在 DI 为目的串地址指针所寻址的存储器单元中去. 指针 DI 将根据 DF 的值进行自动调整.	说明: 1. 其中 OPRD 为目的串符号地址. 2. 本指令不影响标志位. 当不使用操作数时, 可用 STOSB 或 STOSW 分别表示字节串或字串的操作.
SUB	减法指令 SUB(SUBtract) 格式: SUB OPRD1, OPRD2 功能: 两个操作数的相减, 即从 OPRD1 中减去 OPRD2, 其结果放在 OPRD1 中.	说明: 示例 SUB DX, CX SUB [BX+25], AX SUB DI, ALFA[SI]

		SUB CL, 20 SUB DATA1[DI][BX], 20A5H
TEST	测试指令 TEST 格式: TEST OPRD1, OPRD2 功能: 其中 OPRD1、OPRD2 的含义同 AND 指令一样, 也是对两个操作数进行按位的‘与’运算, 唯一不同之处是不将‘与’的结果送目的操作数, 即本指令对两个操作数的内容均不进行修改, 仅是在逻辑与操作后, 对标志位重新置位。	说明: TEST 与 AND 指令的关系, 有点类似于 CMP 与 SUB 指令之间的关系。
WAIT	处理器等待指令 WAIT 格式: WAIT 功能: 本指令将使处理器检测 TEST 端脚, 当 TEST 有效时, 则退出等待状态执行下条指令, 否则处理器处于等待状态, 直到 TEST 有效。	说明: 本指令不影响标志位。
XCHG	数据交换指令 XCHG 格式: XCHG OPRD1, OPRD2 其中的 OPRD1 为目的操作数, OPRD2 为源操作数 功能: 将两个操作数相互交换位置, 该指令把源操作数 OPRD2 与目的操作数 OPRD1 交换。	说明: 1. OPRD1 及 OPRD2 可为通用寄存器或存储器, 但是两个存储器之间是不能用 XCHG 指令实现的。 2. 段寄存器内容不能用 XCHG 指令来交换。 3. 若要实现两个存储器操作数 DATA1 及 DATA2 的交换, 可用以下指令实现: 示例: PUSH DATA1 PUSH DATA2 POP DATA1 POP DATA2 4. 本指令不影响状态标志位。
XLAT	查表指令 XLAT 格式: XLAT TABLE 其中 TABLE 为一待查表格的首地址。 功能: 把待查表格的一个字节内容送到 AL 累加器中。	说明: 1. 在执行该指令前, 应将 TABLE 先送至 BX 寄存器中, 然后将待查字节与在表格中距表首地址位移量送 AL, 即 $(AL) \leftarrow ((BX) + (AL))$。 2. 本指令不影响状态标志位, 表格长度不超过 256 字节。
XOR	逻辑异或运算指令 XOR 格式: XOR OPRD1, OPRD2 功能: 实现两个操作数按位‘异或’运算, 结果送至目的操作数中。 OPRD1 $\leftarrow$ OPRD1 XOR OPRD2	说明: 1. 其在 OPRD1、OPRD2 的含义与 AND 指令相同, 对标志位的影响与 AND 指令相同。 2. 相异为真, 相同为假。